

Autonomous Wheelchair Docking Robot

Rutwik Nakkalla - ME23B113

Nikhita Velangi - ME23B093

Akshay Jha - ME23B127

Rupak Kanti Ghosh - ME23B146

Samdur Tamanna Mohan - ME21B175

May 2025

Contents

1	Aims and Objectives	2
2	Chassis Design and Component Placement	2
2.1	Component List and Justifications	3
3	Electrical connections	3
4	Controller Design and Tuning Method Used	4
4.1	Control System Block Diagram and Transfer Function Model	5
4.2	Code	6
5	Arduino Code for Wall Following and Turning	7
6	Conclusion	9
7	Team Member Contributions	9
8	Videos and photos	9
8.1	Bill	10

1 Aims and Objectives

The aim of this project is to assist people with mobility problems who use a wheelchair and cannot independently place it in the trunk of a car. The objective is to develop a compact, affordable and autonomous robotic system that can transport a wheelchair from the passenger side to the rear of the car with a single button press. This robot must navigate alongside the car maintaining a minimum distance of 1 cm, turn at the rear end, and stop at a precise location. The system avoids vision-based solutions and uses simple sensors and a micro-controller such as Arduino for control.

2 Chassis Design and Component Placement

The robot is designed to fit within a 10x10x10 cm cube. A polycarbonate sheet was chosen as the chassis material because of its durability and low weight. We cut the chasis so that including the wheels, the total size is less than or equal to 10 cm. The wheels and motors are mounted symmetrically to balance the load and ensure maneuverability. The caster wheel has been placed at the back of the chasis such that all the wheels align properly. All connections to the motor are soldered. The Arduino UNO is placed centrally against the batteries' holder to minimize wire clutter. Lithium ion batteries are used as they are compact and rechargeable power source with adequate voltage and current rating. The motor driver is mounted on the Arduino. The ultrasonic sensor is placed at the front for optimal side-distance sensing. Components are securely fixed with double-sided tape and some screws to prevent vibration-induced errors. Various positions of the components have been tested and came up with the best arrangement without causing any damage to the other components, and it works efficiently. The design decisions were made by practicality, availability, minimalism, and ease of construction.

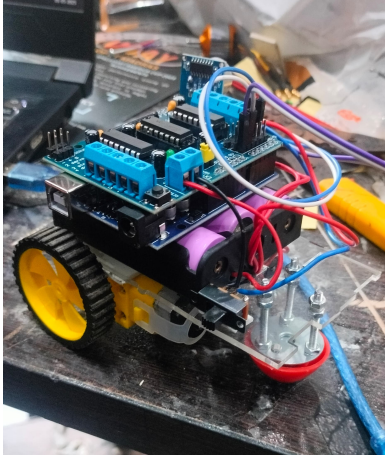


Figure 1: Previous arrangement

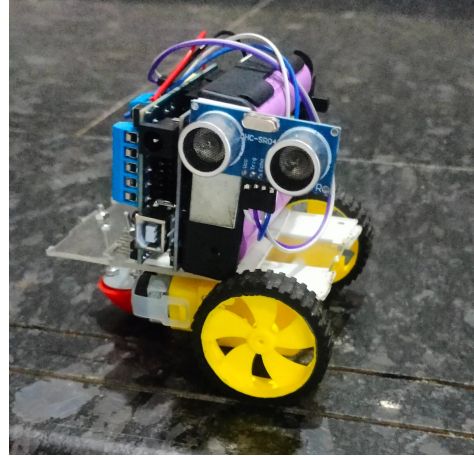


Figure 2: Final arrangement

2.1 Component List and Justifications

Table 1: Component list and justifications

Component	Quantity	Working range	Justification
Arduino UNO	1	5V Power supply	Compact board suitable for integration within the limited space of the bot.
Lithium-ion Battery and its holder	1	3.7V to 4.2V	Chosen for its small form factor and ability to fit easily within the confined chassis.
L293D Motor Driver	1	4.5V to 36V	Integrated IC design ensures minimal space usage compared to bulkier motor drivers.
HC-SR04 Ultrasonic Sensor	1	2 cm to 400 cm	Compact enough to mount on the bot while still providing accurate distance measurements.
DC Motors	2 or 3	3V to 12V	Medium-sized motors that provide necessary torque without exceeding space limits.
Slide Switch	1	On/Off	Very small size; easily fits on the chassis for manual control access.
Polycarbonate Sheet	As required	-	Cut and shaped to fit within the 10cm $\times$ 10cm $\times$ 10cm bounding box constraint.
Jumper Wires, Electrical Tape	Various	-	Flexible and lightweight, ideal for compact wiring within tight spaces.
Wheels	3	-	compact caster wheel is selected to ensure smooth movement while staying within the bot's size restrictions .

3 Electrical connections

The connections are shown in the Fig. 3. The electrical setup is designed for a compact and efficient layout within the 10 cm cube chassis. Here is a detailed explanation of the connections:

- The **Arduino UNO** is the central microcontroller, powered by a **Lithium-ion battery** through a regulated power supply.
- Two **DC motors** are connected to the **L293D Motor Driver**, which is responsible for driving the motors. The motor driver's input pins (IN1, IN2, IN3, IN4) are connected to digital output pins of the Arduino (e.g., D3, D4, D5, D6).

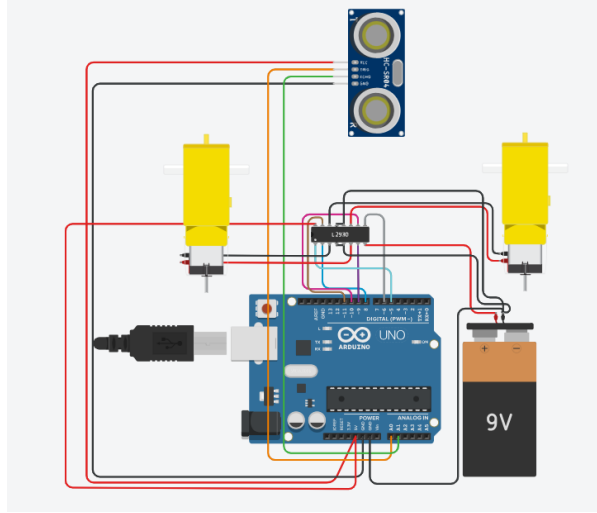


Figure 3: Connections

- The motor driver's power input (VCC) is supplied with 6V from the battery, and the logic power (VSS) is connected to the 5V output of the Arduino.
- The **HC-SR04 Ultrasonic Sensor** is mounted at the front of the robot and connected as follows:
 - VCC to 5V (Arduino)
 - GND to GND
 - TRIG to digital pin A0
 - ECHO to digital pin A1
- A **slide switch** is connected between the battery and the Arduino to allow manual power control.
- A **common ground** is shared between the battery, Arduino, motor driver, and sensor to ensure signal integrity.

All connections are soldered and fixed using jumper wires and securely mounted using insulation tape to avoid disconnections due to vibration. The layout was optimized after multiple iterations for minimal wire overlap and robustness in operation.

4 Controller Design and Tuning Method Used

The transfer function of a typical first-order robotic response system is:

$$G(s) = \frac{K}{\tau s + 1} \quad (1)$$

Where:

- K is the system gain
- τ is the system time constant

The robotic system used in this project can be effectively modeled as a first-order system because its dynamic behavior is dominated by inertial lag without significant oscillatory effects. The motors used exhibit first-order response characteristics at low speeds, where the output (robot motion) changes smoothly in response to input signals without overshoot. The ultrasonic sensor used for distance feedback introduces measurement delay but not reactive dynamics, further supporting a lag-dominated response. Since the system involves a single control loop with a single actuator and feedback source and lacks elastic or spring-like components, its overall response to control inputs resembles that of a standard first-order system characterized by a single time constant and exponential approach to steady state. This makes the first-order approximation both practical and sufficiently accurate for initial modeling and PID controller design.

The unity feedback has been taken. In mobile robotics and Arduino-based systems, unity feedback is standard when designing basic feedback loops using sensors and actuators. The gain and τ are taken to be 1 for the simplified analysis.

A PID controller (Proportional–Integral–Derivative) is used to control the trajectory of the robot relative to the car’s side. It calculates an error signal from the difference between desired and measured distances (via the ultrasonic sensor) and adjusts the motor speeds and directions accordingly. The values of K_p, K_d and K_I were determined through experimental trials by manually adjusting and observing the system’s response. MATLAB was not used for tuning the transfer function in this process.

The PID controller is modeled as:

$$C(s) = K_p + \frac{K_i}{s} + K_d s \quad (2)$$

Tuning is performed to obtain optimal K_p , K_i , and K_d values that minimize steady-state error and improve response time. Through experiment, the obtained values of K_p , K_d and K_I are 0.9, 7, 0 respectively. The K_I term is set to zero as it has no impact at least in the trials that have been done.

4.1 Control System Block Diagram and Transfer Function Model

Based on the experimental tuning and the assumed system model, the final closed-loop transfer function of the robot is given by:

$$T(s) = \frac{7s + 0.9}{8s + 1.9} \quad (3)$$

This transfer function represents the system’s dynamic response with the applied PID controller parameters. The numerator corresponds to the proportional and derivative effects, while the denominator includes the effects of the system time constant and controller tuning. This confirms a stable and responsive system behavior as observed in the step response Fig. 5.

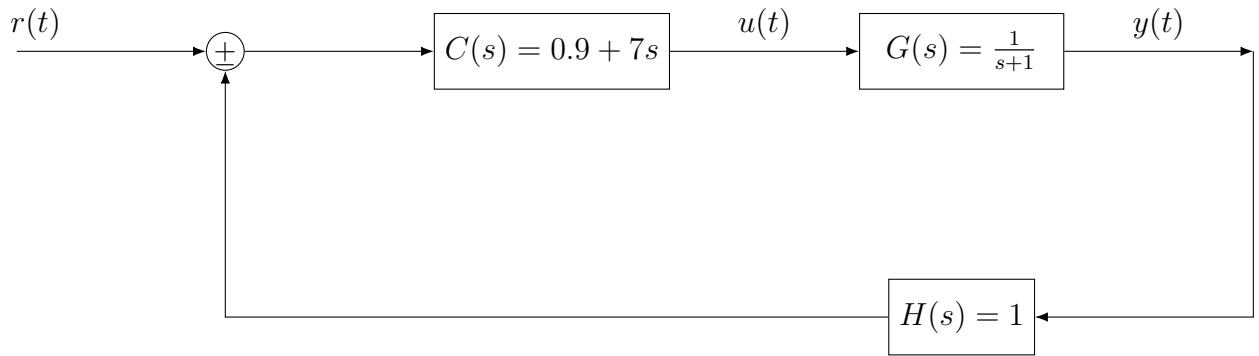


Figure 4: Control system block diagram with PID controller and unity feedback

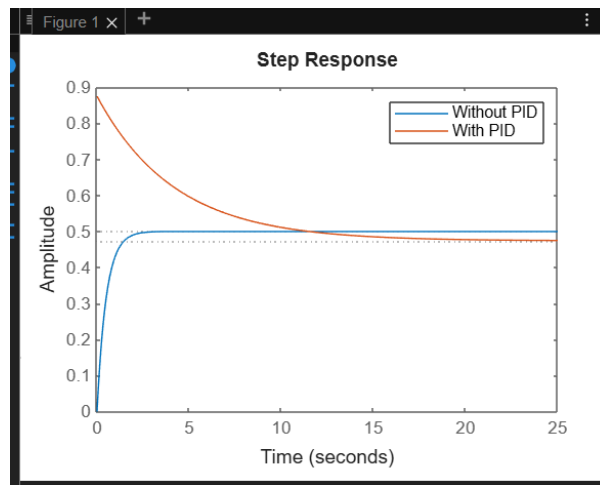


Figure 5: Response of the system before and after tuning

4.2 Code

```

untitled.m x +
/MATLAB Drive/untitled.m
1 clear all
2 clc
3
4 sys = tf([1],[1 1])
5 H = [1]
6
7 Mc = feedback(sys, H)
8
9
10 kp = 0.9
11 kd = 7
12 ki = d
13
14 sys1 = pid(kp, ki, kd)
15 Mc = feedback( sys1*sys, H)
16 step(H)
17 hold on
18 step(Mc)
19 legend('Without PID', 'With PID')
20

```

Figure 6: Matlab code for the response

The code used to run the program:

5 Arduino Code for Wall Following and Turning

```
1 #include <AFMotor.h>
2 // Motor
3 AF_DCMotor motorLeft(3);    // M3 - L
4 AF_DCMotor motorRight(4);   // M4 - R
5
6 // Ultrasonic sensor pins
7 const int trigPin = A0;
8 const int echoPin = A1;
9
10 const float targetDistance = 5.0;    // Target wall d
11 const float Kp = 0.9;                // Kp
12 const float Kd = 7.0;                // Kd
13
14 float distance;
15 float lastDistance = targetDistance;
16 unsigned long lastLoopTime = 0;
17 const unsigned long loopIntervalMs = 50;
18
19 bool inWallFollowMode = true;
20 bool turning = false;
21 int turnStep = 0;
22 unsigned long turnStartTime = 0;
23 unsigned long turnDurations[] = {250, 200, 250, 500, 300};
24
25 // Setup
26 void setup() {
27     Serial.begin(9600);
28     pinMode(trigPin, OUTPUT);
29     pinMode(echoPin, INPUT);
30     motorLeft.setSpeed(80);
31     motorRight.setSpeed(80);
32     motorLeft.run(BACKWARD);
33     motorRight.run(BACKWARD);
34 }
35
36 // Main loop
37 void loop() {
38     unsigned long currentTime = millis();
39
40     if (inWallFollowMode && !turning && currentTime - lastLoopTime
41         >= loopIntervalMs) {
42         lastLoopTime = currentTime;
43         digitalWrite(trigPin, LOW);
44         delayMicroseconds(2);
45         digitalWrite(trigPin, HIGH);
46         delayMicroseconds(10);
47         digitalWrite(trigPin, LOW);
48
49         long duration = pulseIn(echoPin, HIGH, 20000);
```

```

49     distance = duration * 0.0344 / 2;
50     Serial.print("Distance from wall: ");
51     Serial.print(distance);
52     Serial.println(" cm");
53
54     if (distance > 100) {
55         Serial.println("Wall lost! Executing 90 turn...");
56         inWallFollowMode = false;
57         turning = true;
58         turnStep = 0;
59         turnStartTime = currentTime;
60     } else {
61         float error = targetDistance - distance;
62         float derivative = (distance - lastDistance) / (
            loopIntervalMs / 1000.0);
63         int correction = Kp * error - Kd * derivative;
64         correction = constrain(correction, -30, 30);
65
66         int baseSpeed = 80;
67         int leftSpeed = constrain(baseSpeed - correction, 0, 255);
68         int rightSpeed = constrain(baseSpeed + correction, 0, 255);
69
70         motorLeft.setSpeed(leftSpeed);
71         motorRight.setSpeed(rightSpeed);
72         motorLeft.run(BACKWARD);
73         motorRight.run(BACKWARD);
74         lastDistance = distance;
75     }
76 }
77
78 if (turning && currentTime - turnStartTime >= turnDurations[
    turnStep]) {
79     turnStartTime = currentTime;
80     turnStep++;
81
82     switch (turnStep) {
83         case 1:
84             motorLeft.run(RELEASE);
85             motorRight.run(RELEASE);
86             break;
87         case 2:
88             motorLeft.setSpeed(100);
89             motorRight.setSpeed(100);
90             motorLeft.run(BACKWARD);
91             motorRight.run(FORWARD);
92             break;
93         case 3:
94             motorLeft.setSpeed(100);
95             motorRight.setSpeed(100);
96             motorLeft.run(BACKWARD);
97             motorRight.run(BACKWARD);

```

```

98         delay(800);
99         break;
100     case 4:
101         motorLeft.run(RELEASE);
102         motorRight.run(RELEASE);
103         break;
104     case 5:
105         turning = false;
106         inWallFollowMode = true;
107         break;
108     }
109 }
110 }

```

Listing 1: Arduino code for wall following and turning control

6 Conclusion

This project shows how simple sensors and control methods can be used to build a low-cost robot that helps place a wheelchair automatically. By using a basic PID controller, the robot can turn by itself, stop at the right place, and move safely without hitting a model car.

The robot's modular design makes it easy to upgrade for more advanced uses in assistive robotics. Even though the hardware is simple, we were able to achieve accurate and smooth movement by adjusting the PID settings and using a basic control model.

We also made smart design choices, like using a light polycarbonate frame, a small sensor setup, and an Arduino for control. These helped us meet size limits while still keeping everything working well.

Even though this was made in a college environment, it could work in real-life situations with a few small changes. Overall, the project shows that affordable electronic systems and control methods can help improve tools for people with physical disabilities. It's a good starting point for future work in making mobility devices more helpful and user-friendly.

7 Team Member Contributions

- **Rutwik,Akshay,Rupak:** Programming the Arduino, implementing PID control, Tuning the bot, Testing the controller, and debugging the circuit.
- **Nikhita, Tamanna:** Mechanical design, chassis fabrication, component placement, assembling, overall structure balancing and report compiling

8 Videos and photos

[Click here to Redirect to Drive](#)

8.1 Bill

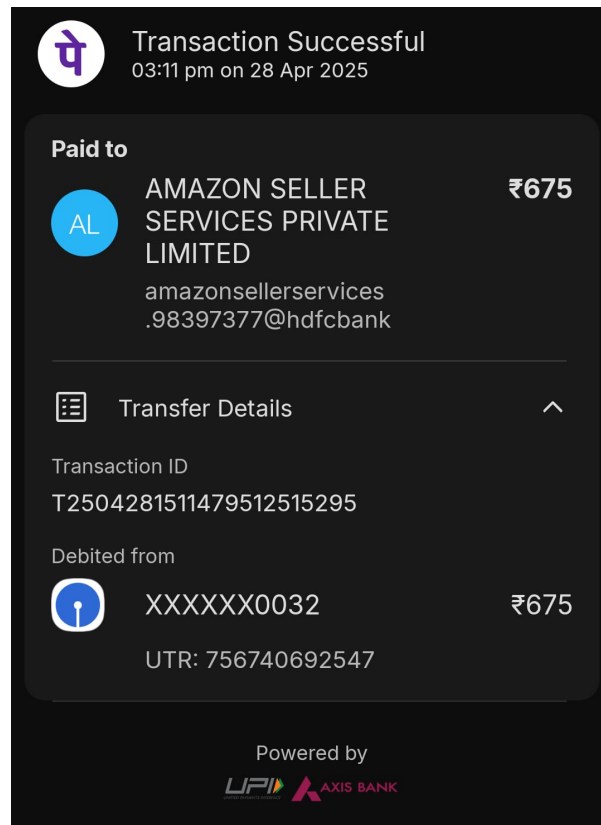


Figure 7: Total bill